BWAI Workshop Guide & Handout

This document contains full preparation guides and hands-on session materials.

Doc: 00-architecture-compatibility-matrix.md

Attendee Laptop Architecture & OS Compatibility Guide (Compatibility Matrix)

When preparing workshop sessions, always verify the tool recommendations and fallback paths below to accommodate attendees' diverse hardware architectures (Apple Silicon, Intel Mac, Windows x86, Windows ARM64, Linux).

Compatibility & Fallback Matrix

| # | Attendee Environment | Recommended Primary Tool | Known Risks & Constraints | Fallback Path |

| --- | --- | --- | --- | --- |

| 1 | **macOS Apple Silicon** (M1-M4) | LM Studio / Ollama | None (Metal GPU acceleration supported) | MLX (`mlx-lm`) available as optional accelerator |

| 2 | **macOS Intel Mac** (x86_64) | **Ollama CLI** | **LM Studio lacks GPU acceleration / freezing reported** | **Provide Ollama migration guide instead of LM Studio** |

| 3 | **Windows x86_64** (Intel/AMD) | LM Studio / Ollama | PowerShell script execution restrictions, WSL2 required for Docker | Run `Set-ExecutionPolicy -Scope Process Bypass` |

| 4 | **Windows ARM64** (Snapdragon) | Ollama CLI | Inference speed degradation under x64 emulation | Use Ollama native ARM64 build |

| 5 | **Linux / ChromeOS** | Ollama CLI | No GUI tool support | Use `ollama serve` in terminal mode |

Important Notice for Intel Mac Attendees

> **Critical**: LM Studio frequently fails to enable GPU acceleration or terminates unexpectedly on Intel CPU-based Macs.

> Intel Mac users should prepare with **Ollama instead of LM Studio**.

1. **Install Ollama**: https://ollama.com/download/Ollama-darwin.zip

2. **Pull models in advance**:

```bash

ollama pull gemma4:e4b

```

3. **Verify local API host**: Ollama automatically binds to `http://localhost:11434` on startup.

Doc: 00-persona-loop-review-report.md

[Loop Engineering] 4-Persona Attendee Review & Audit Report

> Workshop Topic: **Multi-Language Agent Systems with Google ADK (Python, Go, Java, Kotlin, TypeScript)**

> Verified On: 2026-10-30

This document applies the **Loop Engineering** methodology to audit the workshop curriculum, starter/final code, and environment setup across four distinct attendee personas.

---

1. Non-Coder / PM / Designer Persona

• **Readiness Status**: PASS

• **Evaluation**:

• Plain-language architectural overview and zero-setup Google Colab fallback (`output/colab/workshop_starter.ipynb`) allow participation without local terminal friction.

• Key AI concepts (Quantization, Structured Outputs, RAG, Agent Delegation) are paired with intuitive analogies.

---

2. Novice Developer Persona (Junior / Student)

• **Readiness Status**: PASS

• **Evaluation**:

• Automated prerequisite verification scripts (`./scripts/check_env.sh` and `.\scripts\check_env.ps1`) detect missing SDKs and port conflicts before coding begins.

• RAM-tiered model selection (`gemma4:e2b` for 8GB laptops, `gemma4:e4b` for 16GB laptops, `gemini-3.8-flash` for cloud API) prevents Out-Of-Memory crashes.

---

3. Intermediate Developer Persona (Production Engineer)

• **Readiness Status**: PASS

• **Evaluation**:

• Clear `TODO` exercise boundaries in `workshop/01_starter` and complete, runnable reference solutions in `workshop/02_final`.

• Uses modern 2026 SDK patterns (`google-genai >= 2.30.1`, `@google/genai >= 2.29.1`, `gemini-3.8-flash`, `gemini-4-argon`) and type-safe JSON schema validation.

---

4. Senior / Staff Engineer Persona (Systems Architect)

• **Readiness Status**: PASS

• **Evaluation**:

• Includes multi-agent orchestration patterns (`gemini-4-argon` coordinator + `gemini-3.8-flash` workers), reasoning budget control (`thinking_config`), and offline/venue WiFi resilience (`bundle_offline_assets.sh`).

• Zero deprecated API calls or hardcoded credentials.

References

• [Build with AI Seoul 2026](https://github.com/JAICHANGPARK/2026-bwai-seoul)

• [Workshop Harness Repository](https://github.com/JAICHANGPARK/workshop-harness)

Doc: 01-hardware-and-env.md

Hardware & Operating System Environment Setup Guide

This document provides guidance on selecting the appropriate runtime environment based on each attendee's hardware specifications (RAM, CPU/GPU, OS) and API port configurations for workshop sessions.

Local LLM Server API & Port Specification

When integrating local LLMs into Python, Node.js, or Go applications, verify the correct API endpoint and port number for your runtime tool:

| Runtime Tool | Default Port | Base API URL | OpenAI Compatible Endpoint | Default API Key |

| --- | --- | --- | --- | --- |

| **Ollama** | `11434` | `http://localhost:11434` | `http://localhost:11434/v1` | `ollama` (or any string) |

| **LM Studio** | `1234` | `http://localhost:1234` | `http://localhost:1234/v1` | `lm-studio` (or any string) |

> **Port Tip**: If `http://localhost:11434/v1` throws `Connection Refused`, check if `ollama serve` is running. If `http://localhost:1234/v1` fails, ensure the "Local Inference Server" is started inside LM Studio's Developer tab.

Recommended Models & Settings by Hardware Tier

| RAM | Recommended Model | Runtime Tool | Notes |

| --- | --- | --- | --- |

| **8GB** | `gemma4:e2b`, `qwen3:8b`, or `deepseek-r1:8b` | Ollama / LM Studio | Set context length to 2048 or below |

| **16GB** | `gemma4:e4b`, `qwen3:14b`, or `deepseek-r1:14b` | LM Studio / Ollama | Recommended baseline spec for hands-on labs |

| **32GB+** | `gemma4:26b-a4b`, `gemma4:31b`, `qwen3.6:27b`, `qwen3-coder:30b`, or `llama4:scout` | LM Studio / Ollama / MLX | Supports advanced pipelines and multi-agent labs |

> **Note**: The 'B' in parameter count stands for Billion (1,000,000,000). This does not correspond 1:1 with file size in GB. Check the actual download size for 4-bit quantized (Q4_K_M) models.

OS-Specific Guides

1. Apple Silicon Mac (M1/M2/M3/M4)

• **Recommended Tools**: LM Studio, Ollama, MLX

• Metal GPU acceleration is natively supported, enabling smooth local LLM inference.

2. Intel Mac

• **Recommended Tools**: Ollama CLI

• If LM Studio freezes or hangs, use the terminal-based Ollama CLI for CPU/GPU inference as a fallback (`http://localhost:11434`).

3. Windows (Windows 10/11)

• **Recommended Tools**: LM Studio (GUI) or Ollama for Windows

• PowerShell execution policy bypass:

```powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

• WSL2 (Windows Subsystem for Linux 2) setup is required for Docker/RAG labs.

4. Linux / ChromeOS

• **Recommended Tools**: Ollama CLI

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

References

• **Ollama Documentation**: https://ollama.com

• **LM Studio Developer Portal**: https://lmstudio.ai

• **Build with AI Seoul Reference**: [https://github.com/JAICHANGPARK/2026-bwai-seoul](https://github.com/JAICHANGPARK/2026-bwai-golang-seoul)

Doc: 02-prerequisites.md

Workshop Prerequisites & SDK / API Key Setup Guide

To avoid download delays and configuration errors caused by venue WiFi congestion, **please complete all language, SDK, and API Key setup before the event**.

---

1. Google AI Studio & Gemini API Key Setup

Individual developers and workshop attendees can obtain a free Gemini API Key in under one minute via **Google AI Studio**.

Issuance Steps

1. **Visit Google AI Studio**: Sign in at https://aistudio.google.com

2. **Create API Key**:

• Click the **"Get API key"** button in the left sidebar

• Select **"Create API key"** (link to a new or existing GCP project)

3. **Copy API Key**: Copy the generated key in `AIzaSy...` format.

4. **Set Local Environment Variable**:

• Create a `.env` file in the workshop project root and save the key:

```bash
GEMINI_API_KEY="AIzaSyYourGeneratedApiKeyHere"
```

> **Critical Security Warning**:

> Your API Key is equivalent to a personal password. **Never commit it to a GitHub repository.**

> Verify that `.gitignore` includes `.env` and `*.json` entries.

---

2. Google Cloud Platform (GCP) Vertex AI Key Setup

For enterprise or GCP-based workshops, connect via the Vertex AI API.

GCP Console Setup Steps

1. **Visit Google Cloud Console**: https://console.cloud.google.com

2. **Enable Vertex AI API**:

• Search for `Vertex AI API` and click **[Enable]**

• Or use Google Cloud SDK terminal:

```bash
gcloud services enable aiplatform.googleapis.com
```

3. **Create Service Account & JSON Key**:

• Navigate to `IAM & Admin` > `Service Accounts`

• Click **[Create Service Account]** (Role: `Vertex AI User`)

• In the Keys tab, select **[Add Key] > [JSON]** to download

4. **Set Local Authentication Environment Variable**:

• **macOS / Linux**:

```bash
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your-gcp-key.json"
```

• **Windows (PowerShell)**:

```powershell
$env:GOOGLE_APPLICATION_CREDENTIALS="C:\path\to\your-gcp-key.json"
```

---

3. Development Language & SDK Installation by OS

Python (3.9+) & `uv` Package Manager

• **macOS**:

```bash
brew install python@3.11
curl -LsSf https://astral.sh/uv/install.sh | sh
```

• **Windows (PowerShell)**:

```powershell
winget install Python.Python.3.11
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

(Note: Ensure "Add Python to PATH" is checked during installation)

• **Linux**:

```bash
sudo apt update && sudo apt install -y python3 python3-pip python3-venv
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Node.js & npm (v20+)

• **macOS**: `brew install node`

• **Windows**: `winget install OpenJS.NodeJS`

• **Linux**: `curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt install -y nodejs`

Flutter SDK, GenUI & Flutter Agent Plugins

1. Install Flutter SDK & Dart

• **macOS**: `brew install --cask flutter`

• **Windows**: `winget install Flutter.Flutter`

• **Linux**: `sudo snap install flutter --classic`

2. Install Official Flutter Agent Skills (`flutter/agent-plugins`)

Equip your AI coding assistant with official Flutter/Dart procedural skills and diagnostics:

```bash
# Option A: Universal Skills CLI
npx skills add flutter/agent-plugins --skill '*' --agent universal --yes

# Option B: Discover bundled package skills from dependencies
dart run skills@ get
```

3. Essential GenAI & GenUI Packages

• **Gemini SDK & Generative UI**: `flutter pub add google_generative_ai genui`

• **On-Device LiteRT**: `flutter pub add google_litert`

Android Development & Google `android` CLI

To streamline environment setup, project scaffolding, and headless emulator management, install the official **Google `android` CLI**:

1. Install `android` CLI

• **macOS Apple Silicon (`arm64`)**:

```bash
curl -fsSL https://dl.google.com/android/cli/latest/darwin_arm64/install.sh | bash
```

• **macOS Intel (`x86_64`)**:

```bash
curl -fsSL https://dl.google.com/android/cli/latest/darwin_x86_64/install.sh | bash
```

• **Linux (`x86_64`)**:

```bash
curl -fsSL https://dl.google.com/android/cli/latest/linux_x86_64/install.sh | bash
```

• **Windows (PowerShell)**:

```powershell
```

curl.exe -fsSL https://dl.google.com/android/cli/latest/windows_x86_64/install.cmd -o "$env:TEMP\i.cmd"; & "$env:TEMP\i.cmd"

```

#### 2. Initialize Android Skills

Initialize the Android CLI and install official Android Agent Skills (`edge-to-edge`, `camerax`, `adaptive`, `navigation-3`, `appfunctions`, `testing-setup`):

```bash

android init

android skills add --all

```

3. Verify Android Environment & Emulator

```bash

# Check SDK configuration and connected devices

android info

# List and launch AVD emulator

android emulator list

android emulator start <avd-name>

```

Google Agent Development Kit (ADK) & Google Agents CLI (`google-agents-cli`)

For autonomous agent and multi-agent workshops, install the official ADK SDK for your language track and the Google Agents CLI skill suite:

• **Python (`google-adk` & `google-agents-cli`)**:

```bash

uv add google-adk google-genai pydantic

uvx google-agents-cli setup

```

• **Go (`google.golang.org/adk/v2`)**:

```bash

go get google.golang.org/adk/v2@latest

```

• **Java 21+ / Kotlin (`com.google.adk:google-adk`)**:

Add `com.google.adk:google-adk:0.5.0` (and `com.google.adk:google-adk-kotlin-core:0.5.0` for Kotlin) in Gradle or Maven.

• **TypeScript / Node.js (`@google/adk`)**:

```bash

npm install @google/adk @google/genai zod

npx skills add google/agents-cli

```

Pre-Download Checklist

1. **Download Ollama / LM Studio & pull models ahead of time**:

```bash

ollama pull gemma4:e4b

```

2. **Sync project dependencies in advance**:

```bash

cd workshop/01_starter

uv sync

```

References

• **Google AI Studio Key Issuance**: https://aistudio.google.com

• **GCP Vertex AI Setup**: https://cloud.google.com/vertex-ai/docs

• **Official Android CLI & Skills**: https://developer.android.com/tools/agents/android-cli

• **Flutter AI Development Guide**: https://docs.flutter.dev/ai/get-started

• **Flutter Package Skills**: https://docs.flutter.dev/ai/package-skills

• **Flutter Agent Plugins**: https://github.com/flutter/agent-plugins

• **A2UI Open Protocol**: https://a2ui.org
```

Doc: 20-faq.md

Troubleshooting & Frequently Asked Questions (FAQ)

Troubleshooting Guide

Q1. Ollama / LM Studio Connection Failure (`Connection refused`)

• **Cause**: Local server process is not running or the port number is incorrect.

• **Fix**:

• Ollama: Verify `ollama serve` is running in terminal and `http://localhost:11434` is accessible

• LM Studio: Check the `Developer` tab to confirm `Local Inference Server` status is `Started` (port: `1234`)

Q2. Out of Memory (OOM) or Model Loading Failure

• **Cause**: Insufficient RAM capacity.

• **Fix**:

• For 8GB RAM laptops, use a smaller quantized model such as `gemma4:e2b` or below.

• Close other resource-heavy applications (browser tabs, additional IDEs).

Q3. Windows PowerShell Security Error (`Execution_Policies`)

• **Cause**: PowerShell script execution policy restriction.

• **Fix**:

```powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Frequently Asked Questions (FAQ)

Q: What if there is no internet connection on the day of the event?

A: If you followed the pre-event setup guide and pre-installed models and packages, the workshop runs 100% offline in your local environment.

Q: I'm an Intel Mac user and model generation is slow in LM Studio.

A: On Intel Mac, Ollama CLI provides faster inference than LM Studio. Test with `ollama run gemma4:e4b`.